

□ Note Dettagliate: Gerarchia di Classi e Gestione Risorse in C

Indice

1. [Display.cs](#)
 2. [Shapes.cs](#)
 3. [Storage.cs](#)
 4. [Program.cs](#)
-

Display.cs

□ Codice Originale

```
class Display {  
    public void Line (Point a, Point b) => Console.WriteLine  
        ($"LINE {a} {b}");  
    public void Arc (Point cen, double radius, double start,  
        double end) => Console.WriteLine ($"ARC {cen} {radius}  
        {start} {end}");  
    public void Text (Point pt, string text) =>  
        Console.WriteLine ($"TEXT {pt} {text}");  
}
```

```
// PROBLEMA: in C# non posso avere più di un base type. Come fare  
// nel caso in cui voglio avere una gerarchia in cui una  
// classe possa avere più base type? -> SOLUZIONE:  
// interface.
```

```
// Le interfacce specificano dei comportamenti, dunque presentano  
// solo metodi e non campi/costruttori.
```

```
// In un'interfaccia i metodi sono automaticamente abstract e  
// public.
```

```
// I metodi non devono essere implementati, ma solo dichiarati.  
// Saranno le classi che implementano l'interfaccia a  
// doverne fornire l'implementazione.
```

```
// Una classe può avere più interfacce.
```

```
interface IDrawable {  
    void Draw (Display d);  
}
```

□ Spiegazione Dettagliata

La Classe Display

```
class Display {  
    public void Line (Point a, Point b) => Console.WriteLine  
        ($"LINE {a} {b}");  
}
```

Cosa fa: - Una classe che **NON disegna veramente** - È uno **stub** (simulatore) per testare il sistema senza una GUI - Riceve coordinate e stampa sulla console i comandi di disegno

Esempio di utilizzo:

```
Display d = new Display();  
Point p1 = new(0, 0);  
Point p2 = new(10, 10);  
  
d.Line(p1, p2);  
// Output console: LINE (0, 0) (10, 10)
```

Metodo Line()

```
public void Line (Point a, Point b) => Console.WriteLine ($"LINE  
    {a} {b}");
```

- **Parametri:** due punti (inizio e fine della linea)
- **Ritorno:** void (non ritorna nulla)
- **Sintassi:** => è una **arrow function** (corpo del metodo in una riga)
- **Output:** Stampa il comando di disegno

Equivalente senza arrow function:

```
public void Line(Point a, Point b)  
{  
    Console.WriteLine($"LINE {a} {b}");  
}
```

Metodo Arc()

```
public void Arc (Point cen, double radius, double start, double  
    end) =>  
    Console.WriteLine ($"ARC {cen} {radius} {start} {end}");
```

- **cen:** centro del cerchio/arco
- **radius:** raggio
- **start:** angolo iniziale (in gradi)
- **end:** angolo finale (in gradi)

Esempio:

```
Point centro = new(5, 5);
d.Arc(centro, 10.0, 0, 360); // Cerchio completo
// Output: ARC (5, 5) 10 0 360
```

Metodo Text()

```
public void Text (Point pt, string text) => Console.WriteLine
    ($"TEXT {pt} {text}");
```

- Stampa testo a una posizione specificata
- Utile per etichette e nomi

□ L'Interfaccia IDrawable

```
interface IDrawable {
    void Draw (Display d);
}
```

Cos'è un'Interfaccia?

Una **interfaccia** è un **contratto** che dice: - "Qualsiasi classe che implementi IDrawable DEVE avere un metodo Draw()" - L'interfaccia NON fornisce l'implementazione, solo la firma

Il Problema che Risolve

```
// □ PROBLEMA: Eredità singola
class Shape : BaseClass1
{
    // Shape NON può ereditare da BaseClass2!
}

// □ SOLUZIONE: Interfacce multiple
abstract class Shape : BaseClass, IDrawable, IStorable
{
    // Shape eredita da UNA classe base
    // MA implementa DUE interfacce
}
```

Come Implementare un'Interfaccia

```
class Circle : Shape, IDrawable
{
    // OBBLIGATORIO: implementare Draw()
    public void Draw(Display d)
    {
        d.Arc(Position, Radius, 0, 360);
    }
}
```

```
    }  
}
```

Se non implementi il metodo → **Errore di compilazione!**

Polimorfismo con Interfacce

```
List<IDrawable> drawables = new();  
drawables.Add(new Circle(new(5, 5), 10));  
drawables.Add(new Rectangle(new(0, 0), 20, 15));  
  
foreach (var obj in drawables)  
{  
    obj.Draw(display); // Chiama il Draw() giusto per ogni tipo!  
}
```

Shapes.cs

□ Codice Originale - Parte 1: Point

```
public readonly struct Point {  
    public Point (double x, double y) => (X, Y) = (x, y);  
    public readonly double X, Y;  
    public override string ToString () => $"({X}, {Y})";  
}
```

□ Spiegazione: Point

Cos'è uno Struct?

```
public readonly struct Point
```

- **struct:** Tipo **valore** (non riferimento come le classi)
- **readonly:** I campi NON possono essere modificati dopo la creazione
- È **immutabile** - una volta creato non cambia

Differenza struct vs class:

```
// Struct (valore) - copia  
Point p1 = new(5, 10);  
Point p2 = p1; // COPIA  
p2 = new(0, 0);  
// p1 è ancora (5, 10) - non cambiato!  
  
// Class (riferimento) - stesso oggetto  
class Person { public string Name; }  
Person person1 = new() { Name = "Alice" };  
Person person2 = person1; // RIFERIMENTO
```

```
person2.Name = "Bob";  
// person1.Name è ora "Bob" - cambiato!
```

Campi Readonly

```
public readonly double X, Y;
```

- **readonly:** Una volta assegnati nel costruttore, non possono essere cambiati
- **Sicurezza:** Evita modifiche accidentali

```
Point p = new(10, 20);  
p.X = 50; // ❌ ERRORE! È readonly
```

Inizializzazione con Arrow Function

```
public Point (double x, double y) => (X, Y) = (x, y);
```

Cosa fa: - Crea una tupla (X, Y) assegnando i parametri - Equivalente a:

```
public Point(double x, double y)  
{  
    X = x;  
    Y = y;  
}
```

Override di ToString()

```
public override string ToString () => $"({X}, {Y})";
```

Perché?

```
Point p = new(5, 10);  
Console.WriteLine(p); // Senza override: "Program.Point"  
// Con override: "(5, 10)"
```

Utilizzo nei messaggi:

```
Console.WriteLine($"LINE {p1} {p2}");  
// LINE (0, 0) (10, 10)
```

📄 Codice Originale - Parte 2: Shape Base

```
//NOTA: questo è un esempio di object hierarchy  
abstract class Shape : IStorable, IDrawable {  
    public Shape () { }  
    public Shape (Point pos) => Position = pos;  
    public Point Position { get; set; }  
}
```

```

    public virtual void Draw (Display d) {
    }

    public virtual void Load (SReader r) => Position = new Point
        (r.GetD (), r.GetD ());

    public virtual SWriter Save (SWriter w) => w.Put (GetType
        ().Name).Put (Position.X).Put (Position.Y);
}

```

□ Spiegazione: Shape

Cos'è una Classe Astratta?

```

abstract class Shape : IStorable, IDrawable

```

- **abstract:** NON può essere istanziata direttamente
- È una **base class** che fornisce funzionalità comune a tutte le forme

```

// □ Non è possibile!
Shape s = new Shape();

```

```

// □ Possibile (Circle estende Shape)
Shape s = new Circle(new(5, 5), 10);

```

Ereditarietà Multipla di Interfacce

```

abstract class Shape : IStorable, IDrawable

```

Shape DEVE implementare: - **Da IStorable:** Save() e Load() - **Da IDrawable:** Draw()

Costruttori

```

public Shape () { } // Costruttore
    senza parametri
public Shape (Point pos) => Position = pos; // Costruttore con
    Position

```

Perché due costruttori?

```

Circle c1 = new Circle(); // Usa
    Shape()
Circle c2 = new Circle(new(5, 5), 10.0); // Usa
    Shape(Point)

```

Proprietà Position

```
public Point Position { get; set; }
```

- **get:** Leggi il valore
- **set:** Modifica il valore
- È una **proprietà auto-implementata**

```
Circle c = new(new(10, 20), 5);  
Console.WriteLine(c.Position);      // (10, 20)  
c.Position = new(0, 0);              // Modifica
```

Metodo Virtual Draw()

```
public virtual void Draw (Display d) {  
  
}
```

- **virtual:** Può essere sovrascritto dalle classi derivate
- Implementazione vuota (le classi derivate forniranno il loro Draw)

```
// In Circle:  
public override void Draw(Display d) => d.Arc(Position, Radius,  
    0, 360);
```

```
// In Rectangle:  
public override void Draw(Display d) {  
    // disegna le 4 linee  
}
```

Metodo Load() - Caricamento da File

```
public virtual void Load (SReader r) => Position = new Point  
    (r.GetD (), r.GetD ());
```

Cosa fa: - Legge due double dal file (X e Y della posizione) - Crea un nuovo Point - Lo assegna a Position

Dal file legge:

```
10.5      <- r.GetD() -> X  
20.3      <- r.GetD() -> Y
```

Risultato:

```
Position = new Point(10.5, 20.3)
```

Metodo Save() - Salvataggio su File

```
public virtual SWriter Save (SWriter w) =>
    w.Put (GetType ().Name).Put (Position.X).Put (Position.Y);
```

Cosa fa: - Scrive il nome della classe (es: "Circle", "Rectangle") - Scrive X della posizione - Scrive Y della posizione

Ritorna w per permettere **method chaining**

```
// Senza chaining:
SWriter w = new SWriter("file.txt");
w.Put("Circle");
w.Put(10.5);
w.Put(20.3);

// Con chaining:
new SWriter("file.txt").Put("Circle").Put(10.5).Put(20.3);
```

GetType().Name:

```
Circle c = new();
string name = c.GetType().Name; // "Circle"
```

```
Rectangle r = new();
name = r.GetType().Name; // "Rectangle"
```

Utile perché la classe base NON sa quale tipo di forma sta salvando!

□ Codice Originale - Parte 3: Circle

```
class Circle : Shape {
    public Circle () { }
    public Circle (Point pos, double radius) : base (pos)
        { Radius = radius; }
    public double Radius { get; set; }

    public override void Load (SReader r) {
        base.Load (r);
        Radius = r.GetD ();
    }
    public override SWriter Save (SWriter w) => base.Save
        (w).Put (Radius);
    public override void Draw (Display d) => d.Arc (Position,
        Radius, 0, 360);
}
```

□ Spiegazione: Circle

Eredità da Shape

```
class Circle : Shape
```

Circle **eredita** da Shape: - Riceve Position - Riceve Draw(), Load(), Save()
- Può **sovrascrivere** questi metodi

Costruttori

```
public Circle () { }  
public Circle (Point pos, double radius) :  
    base (pos) { Radius = radius; }
```

Costruttore 1 (senza parametri):

```
Circle c = new();  
// Position = Point(0, 0) (default)  
// Radius = 0 (default)
```

Costruttore 2 (con parametri):

```
Circle c = new(new(5, 5), 10.0);
```

Cosa fa : base(pos):

```
: base (pos) // Chiama il costruttore della classe base con pos
```

È equivalente a:

```
public Circle(Point pos, double radius)  
{  
    Position = pos; // Dalla classe base  
    Radius = radius;  
}
```

Proprietà Radius

```
public double Radius { get; set; }
```

- Specifico di Circle
- Non esiste in Shape

```
Circle c = new(new(0, 0), 5);  
Console.WriteLine(c.Radius); // 5  
c.Radius = 10; // Modifica
```

Override di Load()

```
public override void Load (SReader r) {  
    base.Load (r);  
    Radius = r.GetD ();  
}
```

Cosa fa: 1. Chiama `base.Load(r)` → Carica Position (legge X e Y) 2. Legge il Radius

Dal file:

```
Circle      <- Nome classe (letto da Drawing)  
10.5        <- X (base.Load)  
20.3        <- Y (base.Load)  
5.7         <- Radius (questo metodo)
```

Perché `base.Load()`? Non vogliamo riscrivere il codice per caricare Position (già in Shape).

Override di Save()

```
public override SWriter Save (SWriter w) => base.Save (w).Put  
    (Radius);
```

Cosa fa: 1. Chiama `base.Save(w)` → Scrive il nome della classe e Position 2. Aggiunge il Radius

Nel file:

```
Circle      <- da base.Save()  
10.5        <- X da base.Save()  
20.3        <- Y da base.Save()  
5.7         <- Radius (questo metodo)
```

Method Chaining:

```
base.Save(w)    // Ritorna w  
    .Put(Radius) // Usa w ritornato
```

Override di Draw()

```
public override void Draw (Display d) =>  
    d.Arc (Position, Radius, 0, 360);
```

- **Position:** Centro del cerchio
- **Radius:** Raggio
- **0, 360:** Angolo da 0° a 360° (cerchio completo)

Visualizzazione:

```
Circle c = new(new(5, 5), 10);
c.Draw(display);
// Output: ARC (5, 5) 10 0 360
```

❑ Codice Originale - Parte 4: Rectangle

```
class Rectangle : Shape {
    public Rectangle () { }
    public Rectangle (Point pos, double width, double height) :
        base (pos) { (Width, Height) = (width, height); }
    public double Width { get; set; }
    public double Height { get; set; }

    public override void Load (SReader r) {
        base.Load (r);
        Width = r.GetD ();
        Height = r.GetD ();
    }

    public override SWriter Save (SWriter w) => base.Save
        (w).Put (Width).Put (Height);

    public override void Draw (Display d) {
        Point br = new Point (Position.X + Width, Position.Y);
        Point tr = new Point (br.X, br.Y+ Height);
        Point tl = new Point (Position.X, tr.Y);
        d.Line (Position, br); d.Line (br, tr);
        d.Line (tr, tl); d.Line (tl, Position);
    }
}
```

❑ Spiegazione: Rectangle

Tuple Assignment

```
: base (pos) { (Width, Height) = (width, height); }
```

Cosa fa:

```
(Width, Height) = (width, height);
```

Equivalente a:

```
Width = width;
Height = height;
```

È una **tuple assignment** moderna in C#.

Override di Load()

```
public override void Load (SReader r) {
    base.Load (r);
    Width = r.GetD ();
    Height = r.GetD ();
}
```

Dal file:

```
Rectangle    <- Nome (letto da Drawing)
10           <- X (base.Load)
15           <- Y (base.Load)
20           <- Width (questo metodo)
12           <- Height (questo metodo)
```

Override di Save()

```
public override SWriter Save (SWriter w) => base.Save (w).Put  
    (Width).Put (Height);
```

Nel file:

Rectangle
10
15
20
12

Override di Draw() - Calcolo dei Vertici

```
public override void Draw (Display d) {
    Point br = new Point (Position.X + Width, Position.Y);
    Point tr = new Point (br.X, br.Y + Height);
    Point tl = new Point (Position.X, tr.Y);
    d.Line (Position, br);
    d.Line (br, tr);
    d.Line (tr, tl);
    d.Line (tl, Position);
}
```

Visualizzazione grafica:

```

Position (10, 15)                br (30, 15)
  tl (10, 27) ----- tr (30, 27)
    |
    |      Width=20
    |      Height=12
    |
  (10, 15) ----- (30, 15)
    br

```

Calcolo dei punti:

```
Point br = new Point(Position.X + Width, Position.Y);  
// br.X = 10 + 20 = 30  
// br.Y = 15 (stesso Y di Position)  
// br = (30, 15)
```

```
Point tr = new Point(br.X, br.Y + Height);  
// tr.X = 30  
// tr.Y = 15 + 12 = 27  
// tr = (30, 27) Top-Right
```

```
Point tl = new Point(Position.X, tr.Y);  
// tl.X = 10  
// tl.Y = 27  
// tl = (10, 27) Top-Left
```

Disegno delle 4 linee:

```
d.Line(Position, br);      // Bottom: (10, 15) -> (30, 15)  
d.Line(br, tr);            // Right: (30, 15) -> (30, 27)  
d.Line(tr, tl);            // Top: (30, 27) -> (10, 27)  
d.Line(tl, Position);      // Left: (10, 27) -> (10, 15)
```

Output console:

```
LINE (10, 15) (30, 15)  
LINE (30, 15) (30, 27)  
LINE (30, 27) (10, 27)  
LINE (10, 27) (10, 15)
```

□ Codice Originale - Parte 5: Drawing

```
class Drawing : IStorable, IDrawable {  
    public Drawing () { }  
    public Drawing (string name) { Name = name; }  
    public string Name { get; set; }  
    public List<Shape> Shapes { get; } = new ();  
  
    public void Draw (Display d) {  
        d.Text (new (0, 0), Name);  
        foreach (var s in Shapes) {  
            s.Draw (d);  
        }  
    }  
  
    public void Load (SReader r) {  
        Name = r.GetS ();  
        int count = r.GetN ();  
    }  
}
```

```

        for (int i = 0; i < count; i++) {
            Shape s = r.GetS () switch {
                "Circle" => new Circle (),
                "Rectangle" => new Rectangle (),
                _ => throw new Exception ("Unknown shape")
            };
            s.Load (r);
            Shapes.Add (s);
        }
        if (r.GetS () != "EOF") throw new Exception ("Read
error");
    }

    public SWriter Save (SWriter w) {
        w.Put (Name).Put (Shapes.Count);
        foreach (Shape s in Shapes)
            s.Save (w);
        return w.Put ("EOF");
    }
}

```

□ Spiegazione: Drawing

Cos'è Drawing?

Una **collezione di forme** con un nome. È il contenitore principale del disegno.

Proprietà

```

public string Name { get; set; }
public List<Shape> Shapes { get; } = new ();

```

- **Name:** Nome del disegno
- **Shapes:** Lista di forme
- Il get di Shapes è pubblico, ma **non c'è set** (non può essere sostituita la lista)

```

Drawing d = new("Figure1");
d.Shapes.Add(new Circle(...)); // □ OK

```

```

d.Shapes = new List<Shape>(); // □ ERRORE! Shapes è readonly

```

Metodo Draw()

```

public void Draw (Display d) {
    d.Text (new (0, 0), Name);
    foreach (var s in Shapes) {
        s.Draw (d);
    }
}

```

```
    }  
}
```

Cosa fa: 1. Stampa il nome del disegno 2. Per ogni forma nella lista, chiama Draw()

Output esempio:

```
TEXT (0, 0) Figure1  
LINE (10, 15) (30, 15)  
LINE (30, 15) (30, 27)  
...  
ARC (2, 5) 10.2 0 360
```

Metodo Save() - Salvataggio Completo

```
public SWriter Save (SWriter w) {  
    w.Put (Name).Put (Shapes.Count);  
    foreach (Shape s in Shapes)  
        s.Save (w);  
    return w.Put ("EOF");  
}
```

Passo 1: Scrive il nome e il numero di forme

```
w.Put (Name).Put (Shapes.Count);
```

Passo 2: Salva ogni forma

```
foreach (Shape s in Shapes)  
    s.Save(w);
```

Passo 3: Scrive "EOF" (End Of File)

```
return w.Put("EOF");
```

Esempio di file generato:

```
Figure1          <- Nome  
3                <- Numero di forme  
Circle           <- Tipo forma 1  
2               <- X (dalla base)  
5               <- Y (dalla base)  
10.2            <- Radius  
Rectangle        <- Tipo forma 2  
10              <- X  
15              <- Y  
20              <- Width  
12              <- Height  
Circle           <- Tipo forma 3  
12              <- X  
14              <- Y
```

```
3.5          <- Radius
EOF          <- Fine file
```

Metodo Load() - Caricamento Completo

```
public void Load (SReader r) {
    Name = r.GetS ();
    int count = r.GetN ();
    for (int i = 0; i < count; i++) {
        Shape s = r.GetS () switch {
            "Circle" => new Circle (),
            "Rectangle" => new Rectangle (),
            _ => throw new Exception ("Unknown shape")
        };
        s.Load (r);
        Shapes.Add (s);
    }
    if (r.GetS () != "EOF") throw new Exception ("Read error");
}
```

Passo 1: Legge il nome

```
Name = r.GetS();
```

Passo 2: Legge il numero di forme

```
int count = r.GetN();
```

Passo 3: Per ogni forma:

```
for (int i = 0; i < count; i++) {
    Shape s = r.GetS() switch {
        "Circle" => new Circle(),
        "Rectangle" => new Rectangle(),
        _ => throw new Exception("Unknown shape")
    };
    s.Load(r);
    Shapes.Add(s);
}
```

Switch Expression - Cosa fa:

```
Shape s = r.GetS() switch {
    "Circle" => new Circle(),
    "Rectangle" => new Rectangle(),
    _ => throw new Exception("Unknown shape")
};
```

- Legge il nome della forma dal file
- **Se "Circle"** → crea una nuova Circle vuota

- **Se "Rectangle"** → crea una nuova Rectangle vuota
- ****Se altro ()**** → lancia un'eccezione

Poi chiama `s.Load(r)` che carica i dati specifici.

Passo 4: Verifica EOF

```
if (r.GetS() != "EOF")
    throw new Exception("Read error");
```

Controlla che il file sia completo.

Storage.cs

□ Codice Originale

```
class SWriter : IDisposable {
    public SWriter (string name) {
        if (File.Exists (name)) File.Delete (name);
        mFile = new StreamWriter (name);
    }
    public SWriter Put (string s) { mFile.WriteLine (s); return this; }
    public SWriter Put (double f) { mFile.WriteLine (f); return this; }
    public SWriter Put (int d) { mFile.WriteLine (d); return this; }
    public void Dispose () {
        if (mFile != null) mFile.Dispose ();
        mFile = null;
    }

    StreamWriter mFile;
}

class SReader : IDisposable {
    public SReader (string name) {
        mFile = new StreamReader (name);
    }
    public void Dispose () {
        if (mFile != null) mFile.Dispose ();
        mFile = null;
    }
    public string GetS () => mFile.ReadLine ();
    public double GetD () => double.Parse (mFile.ReadLine ());
    public int GetN () => int.Parse (mFile.ReadLine ());
    StreamReader mFile;
}
```

```
interface IStorable {
    StreamWriter Save (StreamWriter w);
    void Load (SReader r);
}
```

❑ Spiegazione: Storage

Il Problema: Risorse Non Gestite

```
// ❑ SENZA Dispose
StreamWriter file = new StreamWriter("test.txt");
file.WriteLine("Hello");
// Se il programma crasha prima di chiudere il file...
// Il file rimane APERTO!
```

Conseguenze: - ❑ Memory leak - ❑ Altre applicazioni non possono aprire il file - ❑ Dati non salvati correttamente

La Soluzione: IDisposable

```
public interface IDisposable {
    void Dispose();
}
```

- Definisce il metodo Dispose()
- Chi implementa DEVE fornire il codice per liberare risorse

❑ Classe StreamWriter

```
class StreamWriter : IDisposable {
    public StreamWriter (string name) {
        if (File.Exists (name)) File.Delete (name);
        mFile = new StreamWriter (name);
    }
    public StreamWriter Put (string s) { mFile.WriteLine (s); return this; }
    public StreamWriter Put (double f) { mFile.WriteLine (f); return this; }
    public StreamWriter Put (int d) { mFile.WriteLine (d); return this; }
    public void Dispose () {
        if (mFile != null) mFile.Dispose ();
        mFile = null;
    }

    StreamWriter mFile;
}
```

Costruttore

```
public StreamWriter (string name) {  
    if (File.Exists (name)) File.Delete (name);  
    mFile = new StreamWriter (name);  
}
```

Cosa fa: 1. Controlla se il file esiste 2. Se esiste, lo cancella 3. Crea un nuovo StreamWriter

Perché cancellare? Se il file già esiste e lo apri in scrittura, aggiungerebbe dati. Vogliamo un file pulito.

Esempio:

```
StreamWriter w = new("document.txt");  
// Se document.txt esiste, viene cancellato  
// Crea un nuovo document.txt vuoto
```

Metodi Put() - Method Chaining

```
public StreamWriter Put (string s) { mFile.WriteLine (s); return  
    this; }  
public StreamWriter Put (double f) { mFile.WriteLine (f); return  
    this; }  
public StreamWriter Put (int d) { mFile.WriteLine (d); return this; }
```

Cosa fanno: - Scrivono il valore nel file (una linea per valore) - **Ritornano this** (ritornano se stessi)

Perché ritornare this?

```
// SENZA chaining:  
StreamWriter w = new StreamWriter("file.txt");  
w.Put("Circle");  
w.Put(10.5);  
w.Put(20.3);  
  
// CON chaining:  
new StreamWriter("file.txt")  
    .Put("Circle")      // Ritorna w  
    .Put(10.5)          // Usa w ritornato, ritorna w  
    .Put(20.3);         // Usa w ritornato, ritorna w
```

Overloading: Tre versioni di Put() - una per tipo!

```
w.Put("Circle");      // Chiama Put(string)  
w.Put(10.5);          // Chiama Put(double)  
w.Put(20);            // Chiama Put(int)
```

Metodo Dispose()

```
public void Dispose () {  
    if (mFile != null) mFile.Dispose ();  
    mFile = null;  
}
```

Cosa fa: 1. Controlla se il file è aperto (!= null) 2. Se sì, chiama Dispose() su StreamWriter (chiude il file) 3. Imposta mFile = null (evita accessi futuri)

Importante: Questa è l'implementazione di IDisposable!

□ Classe SReader

```
class SReader : IDisposable {  
    public SReader (string name) {  
        mFile = new StreamReader (name);  
    }  
    public void Dispose () {  
        if (mFile != null) mFile.Dispose ();  
        mFile = null;  
    }  
    public string GetS () => mFile.ReadLine ();  
    public double GetD () => double.Parse (mFile.ReadLine ());  
    public int GetN () => int.Parse (mFile.ReadLine ());  
    StreamReader mFile;  
}
```

Costruttore

```
public SReader (string name) {  
    mFile = new StreamReader (name);  
}
```

- Apre un file in **lettura**
- Crea un StreamReader

Metodo Dispose()

```
public void Dispose () {  
    if (mFile != null) mFile.Dispose ();  
    mFile = null;  
}
```

Stesso identico a SWriter - chiude il file.

Metodi Get - Lettura

```
public string GetS () => mFile.ReadLine ();  
public double GetD () => double.Parse (mFile.ReadLine ());  
public int GetN () => int.Parse (mFile.ReadLine ());
```

GetS() - String:

```
string name = reader.GetS();  
// Legge la riga successiva come string  
  
// File:  
// "Circle"  
// reader.GetS() -> "Circle"
```

GetD() - Double:

```
double radius = reader.GetD();  
// Legge la riga e la converte a double  
  
// File:  
// 10.5  
// reader.GetD() -> 10.5
```

GetN() - Int:

```
int count = reader.GetN();  
// Legge la riga e la converte a int  
  
// File:  
// 3  
// reader.GetN() -> 3
```

□ Interfaccia IStorable

```
interface IStorable {  
    SWriter Save (SWriter w);  
    void Load (SReader r);  
}
```

Cosa definisce: - Qualsiasi classe che implementa IStorable DEVE avere: -
Save(SWriter w) - per salvare su file - Load(SReader r) - per caricare da file

Implementazioni:

```
abstract class Shape : IStorable, IDrawable  
{  
    public virtual SWriter Save(SWriter w) { ... }  
    public virtual void Load(SReader r) { ... }
```

```

}

class Drawing : IStorable, IDrawable
{
    public SWriter Save(SWriter w) { ... }
    public void Load(SReader r) { ... }
}

```

Program.cs

□ Codice Originale

```

class Program {
    static void Main ()
    {
        Drawing dwg = new Drawing ("Figure1");
        dwg.Shapes.Add (new Circle (new (2, 5), 10.2));
        dwg.Shapes.Add (new Rectangle (new (10, 15), 12, 11));
        dwg.Shapes.Add (new Circle (new (12, 14), 3.5));

        // NOTA: using viene utilizzato per garantire che le
        // risorse vengono rilasciate correttamente dopo l'uso.
        // In questo caso, SWriter implementa IDisposable, quindi
        // l'uso di using assicura che il file venga chiuso
        // correttamente dopo la scrittura.
        using (var sw = new SWriter ("C:/Academy/A12/test.dg"))
            dwg.Save (sw);

        using (var sr = new SReader ("C:/Academy/A12/test.dg")) {
            var dwg2 = new Drawing ();
            dwg2.Load (sr);
        }

        Display d = new Display ();
        dwg.Draw (d);
    }

    ////Check downcasting
    //Circle newCircle = new (new (3, 5), 3.5);
    //Shape s1 = newCircle;
    //// Utilizzando l'operatore is : pattern matching casting
    //if (s1 is Rectangle) {
    //    Rectangle r1 = (Rectangle)s1;
    //}
    //// Utilizzando l'operatore as : soft casting
    //Rectangle r5 = s1 as Rectangle;
    //if (r5 != null) {

```

```
        //      r5.Width = 3;  
        //}  
}
```

□ Spiegazione: Program.cs

Fase 1: Creazione del Disegno

```
Drawing dwg = new Drawing ("Figure1");
```

- Crea un nuovo disegno vuoto
- Nome: "Figure1"
- La lista Shapes è vuota

Stato:

```
Drawing: Figure1  
Shapes: []
```

Fase 2: Aggiunta delle Forme

```
dwg.Shapes.Add (new Circle (new (2, 5), 10.2));  
dwg.Shapes.Add (new Rectangle (new (10, 15), 12, 11));  
dwg.Shapes.Add (new Circle (new (12, 14), 3.5));
```

Forma 1: Cerchio - Posizione: (2, 5) - Raggio: 10.2

Forma 2: Rettangolo - Posizione: (10, 15) - Larghezza: 12 - Altezza: 11

Forma 3: Cerchio - Posizione: (12, 14) - Raggio: 3.5

Stato dopo:

```
Drawing: Figure1  
Shapes: [  
    Circle(2, 5, r=10.2),  
    Rectangle(10, 15, 12x11),  
    Circle(12, 14, r=3.5)  
]
```

Fase 3: Salvataggio su File (Con using)

```
using (var sw = new StreamWriter ("C:/Academy/A12/test.dg"))  
    dwg.Save (sw);
```

Cosa fa using:

```
// Step 1: Crea StreamWriter  
var sw = new StreamWriter("C:/Academy/A12/test.dg");  
  
// Step 2: Esegue il codice
```

```
dwg.Save(sw);
```

```
// Step 3: Chiama automaticamente Dispose()  
sw.Dispose(); // Chiude il file!
```

Flusso di salvataggio:

```
dwg.Save(sw)  
└─ w.Put("Figure1")           // Nome disegno  
└─ w.Put(3)                   // Numero di forme  
└─ Forma 1: Circle  
    └─ base.Save(w).Put(Radius)  
        └─ w.Put("Circle")  
        └─ w.Put(2)  
        └─ w.Put(5)  
        └─ w.Put(10.2)  
└─ Forma 2: Rectangle  
    └─ base.Save(w).Put(Width).Put(Height)  
        └─ w.Put("Rectangle")  
        └─ w.Put(10)  
        └─ w.Put(15)  
        └─ w.Put(12)  
        └─ w.Put(11)  
└─ Forma 3: Circle  
    └─ w.Put("Circle")  
    └─ w.Put(12)  
    └─ w.Put(14)  
    └─ w.Put(3.5)  
└─ w.Put("EOF")
```

File generato (test.dg):

```
Figure1  
3  
Circle  
2  
5  
10.2  
Rectangle  
10  
15  
12  
11  
Circle  
12  
14  
3.5  
EOF
```

Perché using è importante:

```
// □ SENZA using - il file potrebbe rimanere aperto
SWriter sw = new SWriter("file.txt");
dwg.Save(sw);
// Se c'è un errore qui, il file NON viene chiuso!

// □ CON using - il file viene SEMPRE chiuso
using (var sw = new SWriter("file.txt"))
{
    dwg.Save(sw);
} // Dispose() viene chiamato anche se c'è un errore!
```

Fase 4: Caricamento da File (Con using)

```
using (var sr = new SReader ("C:/Academy/A12/test.dg")) {
    var dwg2 = new Drawing ();
    dwg2.Load (sr);
}
```

Cosa fa:

```
// Step 1: Crea SReader e apre il file
var sr = new SReader("C:/Academy/A12/test.dg");

// Step 2: Crea un Drawing vuoto
var dwg2 = new Drawing();

// Step 3: Carica i dati dal file
dwg2.Load(sr);

// Step 4: Chiude il file
sr.Dispose();
```

Flusso di caricamento:

```
dwg2.Load(sr)
├─ Name = r.GetS()           // "Figure1"
├─ count = r.GetN()         // 3
├─ Loop 1:
│   ├─ s = r.GetS() switch  // "Circle"
│   ├─ s = new Circle()
│   ├─ s.Load(sr)
│   │   ├─ Position = (2, 5)
│   │   └─ Radius = 10.2
│   └─ Shapes.Add(s)
├─ Loop 2:
│   ├─ s = r.GetS() switch  // "Rectangle"
│   ├─ s = new Rectangle()
│   └─ s.Load(sr)
│       └─ Position = (10, 15)
```

```

    └ Width = 12
    └ Height = 11
    └ Shapes.Add(s)
└ Loop 3:
    └ s = r.GetS() switch // "Circle"
    └ s = new Circle()
    └ s.Load(sr)
        └ Position = (12, 14)
        └ Radius = 3.5
    └ Shapes.Add(s)
└ Verifica r.GetS() == "EOF" // OK!

```

Risultato:

dwg2 è identico a dwg (salvo il nome, che è vuoto)

Fase 5: Visualizzazione

```

Display d = new Display ();
dwg.Draw (d);

```

Cosa fa:

```

d.Text(new(0, 0), "Figure1");

// Per ogni forma in dwg.Shapes:
d.Arc(new(2, 5), 10.2, 0, 360); // Circle 1

d.Line(new(10, 15), new(22, 15)); // Rectangle
d.Line(new(22, 15), new(22, 26));
d.Line(new(22, 26), new(10, 26));
d.Line(new(10, 26), new(10, 15));

d.Arc(new(12, 14), 3.5, 0, 360); // Circle 2

```

Output console:

```

TEXT (0, 0) Figure1
ARC (2, 5) 10.2 0 360
LINE (10, 15) (22, 15)
LINE (22, 15) (22, 26)
LINE (22, 26) (10, 26)
LINE (10, 26) (10, 15)
ARC (12, 14) 3.5 0 360

```

❏ Codice Commentato: Downcasting

```

/////Check downcasting
//Circle newCircle = new (new (3, 5), 3.5);

```

```
//Shape s1 = newCircle;
//// Utilizzando l'operatore is : pattern matching casting
//if (s1 is Rectangle) {
//    Rectangle r1 = (Rectangle)s1;
//}
//// Utilizzando l'operatore as : soft casting
//Rectangle r5 = s1 as Rectangle;
//if (r5 != null) {
//    r5.Width = 3;
//}
```

Cosa mostra questo codice?

Creazione di Circle e assegnazione a Shape:

```
Circle newCircle = new (new (3, 5), 3.5); // Circle concreto
Shape s1 = newCircle;                     // Assegnato a Shape
                                   (base class)
```

Upcast: Circle → Shape (automatico, sempre sicuro)

Pattern Matching Casting con is

```
if (s1 is Rectangle) {
    Rectangle r1 = (Rectangle)s1;
    // Esegui codice specifico per Rectangle
}
```

Cosa fa: - Controlla se s1 è realmente un Rectangle - Se sì, cast esplicito e usa come Rectangle

In questo caso:

s1 è un Circle, non un Rectangle
Quindi l'if è FALSE, il codice dentro NON si esegue

Soft Casting con as

```
Rectangle r5 = s1 as Rectangle;
if (r5 != null) {
    r5.Width = 3;
}
```

Cosa fa: - Tenta di castare s1 a Rectangle - Se riesce, ritorna l'oggetto castato - Se fallisce, ritorna null (non lancia eccezione)

In questo caso:

s1 è un Circle, non un Rectangle
r5 = **null**
L'if è FALSE

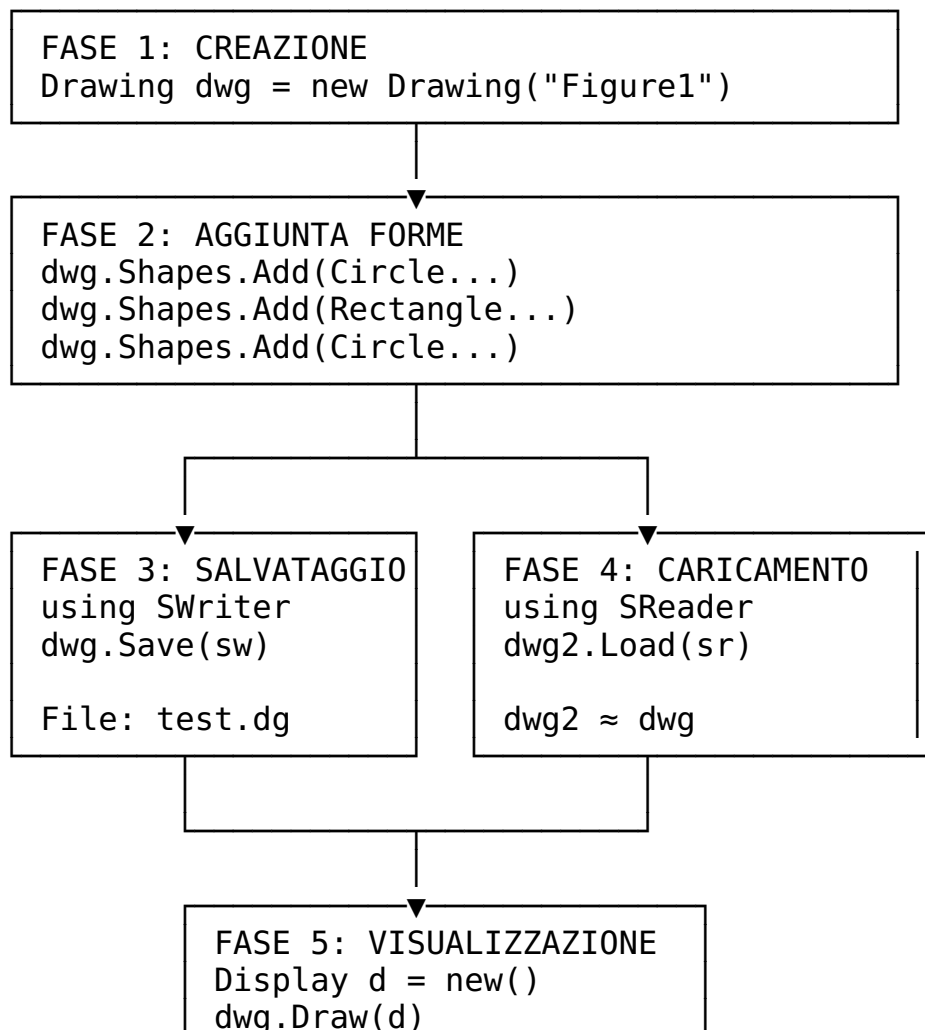
Differenza tra is e as

```
// is: Controlla + Cast
if (obj is Rectangle) {
    Rectangle r = (Rectangle)obj; // Cast esplicito
}

// as: Cast diretto + null check
Rectangle r = obj as Rectangle;
if (r != null) {
    // Usa r
}
```

as è più sicuro (non lancia eccezione se fallisce).

Riassunto Completo del Flusso



```
Output console:  
TEXT (0,0) Figure1  
ARC (2,5) 10.2 0 360  
LINE ...
```

Concetti Chiave Spiegati

1. Interfacce - Contratto di Comportamento

```
interface IDrawable { void Draw(Display d); }  
interface IStorable { SWriter Save(SWriter w); void Load(SReader  
    r); }
```

Una classe che implementa un'interfaccia DEVE fornire tutti i metodi definiti.

2. Classi Astratte - Base per Derivate

```
abstract class Shape : IStorable, IDrawable { ... }
```

Non può essere istanziata, fornisce funzionalità comune.

3. Ereditarietà e Override

```
class Circle : Shape {  
    public override void Draw(Display d) { ... }  
}
```

I metodi virtuali della base class possono essere sovrascritti.

4. Polimorfismo - Stesso Metodo, Comportamenti Diversi

```
foreach (Shape s in shapes) {  
    s.Draw(d); // Circle disegna un arco, Rectangle disegna  
    linee  
}
```

5. IDisposable - Gestione Risorse

```
using (var sw = new SWriter("file.txt")) {  
    // Usa la risorsa  
} // Dispose() chiamato automaticamente
```

6. Method Chaining - Ritornare this

```
w.Put("Circle").Put(10).Put(20);
```

7. Switch Expression - Sintassi Moderna

```
Shape s = name switch {  
    "Circle" => new Circle(),  
    "Rectangle" => new Rectangle(),  
    _ => throw new Exception()  
};
```

Conclusione

Questo progetto dimostra una **architettura software ben strutturata** che utilizza:

□ **OOP avanzato** (interfacce, ereditarietà, polimorfismo) □ **Gestione corretta delle risorse** (IDisposable, using) □ **Persistenza dati** (salvataggio e caricamento) □ **Pattern moderni C#** (method chaining, switch expressions) □ **Codice leggibile e manutenibile**

È un eccellente esempio di come strutturare un progetto C# professionale!